

Django 教學 04

Model與欄位設計

第 6～7 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [03_Template](#)與頁面呈現。

- **第 6 章：Model 與資料身分**
- **第 7 章：欄位型別與資料規則**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [來源索引](#) · [實作手冊對照](#)

第 6 章

Model 與資料身分

在既有 django_lab 加入 catalog，建立分類與商品並在 Admin 找到資料。

本章的操作環境與成果

先在 django_lab 建立 catalog，再依步驟啟動 LearnBoard／LearnMart 對照。

完成成果：在既有 django_lab 加入 catalog，建立分類與商品並在 Admin 找到資料。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

6-1 資料模型銜接：頁面的資料從哪裡來？

Python list 可用來練習顯示；網站需要資料庫保存可持續查詢的資料。

程式中的角色	資料庫對應	商品頁的例子
Model class	資料表	Product
Model instance	一筆資料	某一件商品
Field	欄位	name、price

Model 定義資料，ORM 提供 Python 操作介面，View 決定本次頁面需要什麼。

第 6～11 章學主鍵、欄位、關聯與查詢，第 12～13 章把資料組合成頁面。

6-2 Model 的三種對應

Python 世界	資料庫世界	商品範例
Model class	資料表	Product
Model instance	一筆資料	某一把鍵盤
一般 Field	欄位	name、price
ForeignKey	外鍵欄位	category_id

ORM（物件關聯映射）將 Python 查詢轉為 SQL（資料庫查詢語言），再將結果轉成 Python 物件。

ManyToMany 是例外：它透過中介表保存關係，不是單一欄位。

6-3 一個 Model 的完整骨架

現有 LearnMart | Category，省略中文欄位名稱

```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)
    slug = models.SlugField(max_length=80, unique=True)

    class Meta:
        ordering = ["name"]
        verbose_name = "商品分類"
        verbose_name_plural = "商品分類"

    def __str__(self):
        return self.name
```

欄位描述資料；Meta 描述模型設定；方法描述物件行為。

6-4 `__str__`：物件顯示的文字

放在 Category 類別內的方法：

```
def __str__(self):  
    return self.name
```

- self 是目前這筆物件；return 必須回傳字串。
- print(category)、Admin 關聯選單會使用這段顯示文字。
- 沒定義時常只看到 Category object (1)，不容易辨認資料。
- 這是 Python 方法，不會額外建立資料庫欄位。

不要在這裡任意改資料；若讀取其他關聯，也可能增加查詢。

6-5 Primary key：資料的穩定身分

同名商品可能有很多筆，名稱不能可靠地辨識「是哪一筆」。

```
product = Product.objects.get(pk=17)
print(product.pk, product.id)
```

- 兩個專案使用自動 `BigAutoField` 主鍵；不必手動宣告 `id`。
- `pk` 是主鍵的通用別名；主鍵不叫 `id` 時仍能用 `pk`。
- 主鍵用來定位與建立關聯，不代表排序名次，也不保證連號。
- 不要以「目前筆數 + 1」自行產生主鍵。

6-6 自訂主鍵與 UUID

教學延伸 | 一般情況先保留自動 id

```
import uuid
from django.db import models

class Ticket(models.Model):
    id = models.UUIDField(
        primary_key=True, default=uuid.uuid4, editable=False,
    )
    title = models.CharField(max_length=120)
```

`primary_key=True` 隱含唯一且不可為 NULL；設定後不再另加自動 id。

`uuid.uuid4` 是 callable，不加括號；每次建立才產生新值。

不要把可變動的名稱當主鍵，也不要靠 UUID 取代權限檢查。

6-7 Primary key、unique、複合唯一

設計	回答的問題	例子
主鍵	這一系列是誰？	Category.id
<code>unique=True</code>	此欄位值能否重複？	Category.slug
<code>UniqueConstraint</code>	這組欄位能否重複？	CartItem(user, product)

同一張表可以有多個 unique 欄位；它們不會都變成主鍵。

兩個欄位各自 unique，不等於兩個欄位組合 unique。

進階：複合主鍵另有 `CompositePrimaryKey`，不是在多個欄位都寫 `primary_key=True`；本課沿用單欄位主鍵＋複合唯一約束。

6-8 用資料列理解複合唯一

user_id	product_id	quantity	是否可同時存在？
1	10	2	可以
1	11	1	可以：同買家不同商品
2	10	1	可以：不同買家同商品
1	10	3	不可以：與第一列同一組鍵

現有 CartItem 保留一列，修改 quantity；不是多插入一列。
稍後在 `Meta.constraints` 寫出這條規則。

6-9 延續 django_lab：加入資料練習 app

回到第 1～5 章的 **django_lab/**，保留 config 與 pages：

```
uv add pillow
uv run python manage.py startapp catalog
```

- 不再執行 `uv init` 或 `startproject`，也不另建虛擬環境。
- pages 保留 HTTP 與 Template 練習，catalog 負責分類與商品資料。
- 下一頁註冊 app，再建立完整的 Category/Product。
- 後面的「現有 LearnMart」片段用來對照，勿覆蓋 catalog 的完整範例。

6-10 把 app 加入設定

開啟 config/settings.py，在原 INSTALLED_APPS 清單最後加入：

```
"catalog.apps.CatalogConfig",
```

保留原有 admin、auth、sessions 等項目；先用預設 SQLite 練習。

```
uv run python -m django --version  
uv run python manage.py check
```

若找不到 catalog，先檢查工作目錄、拼字與 app 是否建立成功。

6-11 最小資料練習的完整操作路線

接下來先帶做一輪，再逐項解釋：

1. 在 `catalog/models.py` 建立 `Category` 與 `Product`。
2. `makemigrations`、`migrate`、`showmigrations` 確認資料表。
3. `shell` 建立第一筆分類與商品。
4. Admin 註冊模型、建立管理帳號並查看資料。
5. 設定 `media`，從後台觀察圖片路徑與檔案。

第 7～11 章再展開欄位、關聯、查詢與後台；此處可先把參數當作待解釋的設定。

6-12 最小 models.py：分類

用以下內容開始 catalog/models.py。下一頁接在本頁後面。

```
from django.db import models

class Category(models.Model):
    name = models.CharField("分類名稱", max_length=80, unique=True)
    slug = models.SlugField("網址代稱", max_length=80, unique=True)

    class Meta:
        ordering = ["name"]
        verbose_name = "商品分類"
        verbose_name_plural = "商品分類"

    def __str__(self):
        return self.name
```

這是獨立的 catalog 範例；稍後逐項解釋欄位、Meta 與 `__str__`。

6-13 最小 models.py：商品

接在同一檔案的 Category 後面；不需要另外建立賣家或帳號資料。

```
class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.PROTECT,
                                related_name="products")
    name = models.CharField("商品名稱", max_length=150)
    description = models.TextField("商品說明", blank=True)
    price = models.DecimalField("售價", max_digits=8, decimal_places=2)
    stock = models.PositiveIntegerField("庫存", default=0)
    image = models.ImageField(upload_to="products/", blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

先保留最小資料；分類與商品的一對多將在第 8 章展開。

6-14 建表：migration 是資料結構的版本紀錄

```
uv run python manage.py makemigrations catalog
uv run python manage.py migrate
uv run python manage.py showmigrations catalog
```

第一個命令應產生 catalog/migrations/0001_initial.py。

第二個命令把資料結構套進資料庫；第三個應看到 [X] 0001_initial。

打開 migration，找出 Category、Product 與自動 id；不需手改檔案。

成功判斷不是「指令有輸出」，而是資料表建立且版本顯示已套用。

6-15 Shell：建立第一筆資料

先執行 `uv run python manage.py shell`，再逐行輸入：

```
from decimal import Decimal
from catalog.models import Category, Product
category = Category(name="文具", slug="stationery")
category.full_clean()
category.save()
product = Product(category=category, name="筆記本",
                  price=Decimal("50.00"), stock=3)
product.full_clean()
product.save()
print(product.pk, product.name, product.category.name)
```

成功會看到主鍵、筆記本、文具。先在全新資料庫執行一次；重跑的分類會撞 unique。

6-16 最小 Admin：把資料放到後台

在 catalog/admin.py 填入：

```
from django.contrib import admin
from .models import Category, Product
```

```
admin.site.register(Category)
admin.site.register(Product)
```

```
uv run python manage.py createsuperuser
uv run python manage.py runserver
```

登入 [本機 Admin](#)，新增一件商品並選擇「文具」。

6-17 圖片顯示所需的開發設定

在 config/settings.py 加入：

```
MEDIA_URL = "/media/"  
MEDIA_ROOT = BASE_DIR / "media"
```

MEDIA_ROOT 是本機儲存目錄；MEDIA_URL 是瀏覽器存取的網址前綴。
上傳的圖片屬於 media；網站自己的 CSS、圖示屬於 static，兩者分開。

6-18 開發環境的 media 路由

在 config/urls.py 保留原 urlpatterns，接上：

```
from django.conf import settings
from django.conf.urls.static import static

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)
```

可在 Admin 上傳圖片後開啟檔案連結；此服務方式只供開發環境。

6-19 現在才打開 LearnBoard × LearnMart

先對照自己的 `django_lab`，辨認相同職責的檔案：

角色	你的練習	LearnBoard	LearnMart
外層資料夾	<code>django_lab/</code>	<code>learnboard/</code>	<code>learnmart/</code>
Django project	<code>config/</code>	<code>config/</code>	<code>config/</code>
功能 app	<code>pages/</code>	<code>board/</code>	<code>marketplace/</code>
第一個頁面	打招呼文字	留言列表	商品列表
根路由轉交	<code>pages.urls</code>	<code>board.urls</code>	<code>marketplace.urls</code>

完成專案多了 `model`、`form`、`template`、權限與測試，逐步找相同角色即可。
本教材前半段的 `hello` / `search` 是練習範例，沒有加進完成專案。

6-20 教材根目錄與 Django 應用根目錄

```
DjangoTeaching/           目前整份教材的 Git repository
├── .git/
├── slides/
│   └── courses/           01~17 連續教學投影片
├── learnboard/            執行 LearnBoard 指令的位置
│   ├── pyproject.toml
│   └── manage.py
└── learnmart/             執行 LearnMart 指令的位置
    ├── pyproject.toml
    └── manage.py
```

日常說的「專案」可能指整份版本庫，也可能指一個應用資料夾。

本教材目前由外層 `.git/` 管理，兩個應用不用再各自 `git init`。

執行 Django 指令時，先確認當前目錄同時有 `manage.py` 與 `pyproject.toml`。

6-21 環境對照：共同版本與 Pillow 差異

實際檔案：`learnboard/pyproject.toml` 與 `learnmart/pyproject.toml`。

項目	LearnBoard	LearnMart
<code>.python-version</code>	<code>3.14.7</code>	<code>3.14.7</code>
<code>requires-python</code>	<code>>=3.14,<3.15</code>	<code>>=3.14,<3.15</code>
Django	<code>django>=6.1.1,<6.2</code>	<code>django>=6.1.1,<6.2</code>
圖片套件	無 Pillow 依賴	<code>pillow>=11.0</code>
開發群組	<code>coverage>=7.6</code>	<code>coverage>=7.6</code>
uv 設定	<code>package = false</code>	<code>package = false</code>

LearnBoard 沒有圖片上傳功能，LearnMart 的 `ImageField` 需要 Pillow。
兩者各有自己的 `.venv/` 與 `uv.lock`，不要互相複製虛擬環境。

6-22 實際 TOML 節錄可以這樣讀

LearnBoard 的 `[project]` 依賴節錄：

```
[project]
```

```
name = "learnboard"
```

```
requires-python = ">=3.14,<3.15"
```

```
dependencies = ["django>=6.1.1,<6.2"]
```

LearnMart 的 `[project]` 依賴節錄：

```
[project]
```

```
name = "learnmart"
```

```
requires-python = ">=3.14,<3.15"
```

```
dependencies = ["django>=6.1.1,<6.2", "pillow>=11.0"]
```

這是兩份檔案的節錄，不能把兩個 `[project]` 合貼到同一份檔案。

6-23 settings 對照：先找共同設定

設定	兩個完成專案的共同做法
<code>ROOT_URLCONF</code>	<code>config.urls</code>
<code>DATABASES</code>	SQLite，應用根目錄的 <code>db.sqlite3</code>
<code>TEMPLATES</code>	<code>DIRS</code> 指定根 <code>templates/</code> ， <code>APP_DIRS=True</code>
語言／時區	<code>zh-hant</code> 、 <code>Asia/Taipei</code>
靜態資源	<code>STATICFILES_DIRS = [BASE_DIR / "static"]</code>
本機開發	<code>DEBUG=True</code> ，Host 允許 localhost 與 127.0.0.1

內建 middleware、密碼驗證器與入口檔案也能找到相同職責。
兩份 `SECRET_KEY` 都是課堂示例，不能沿用到公開正式環境。

6-24 settings 對照：功能增加後的差異

項目	LearnBoard	LearnMart
App 註冊	<code>board</code>	<code>marketplace</code>
User model	Django 預設 User	<code>AUTH_USER_MODEL = "marketplace.User"</code>
登入／登出導向	<code>board:list</code>	<code>marketplace:home</code>
上傳檔案	無 media 設定	<code>MEDIA_URL</code> 、 <code>MEDIA_ROOT</code>
額外 context processor	無購物車處理	<code>marketplace.context_processors.cart_count</code>

兩者 `LOGIN_URL` 都是 `login`，`MESSAGE_TAGS` 把錯誤訊息標籤對應為 `danger`。

自訂 User 已在完成專案的 migration 設計內，照既有檔案執行即可。

不要在已建立資料表的練習隨意切換 `AUTH_USER_MODEL`。

6-25 建立指令：完成專案只讀取，不再重跑

對照你剛才使用的指令，理解兩個完成專案的骨架建立方式：

```
# LearnBoard 骨架的等價建立指令 (只供對照)
```

```
uv run django-admin startproject config .
```

```
uv run python manage.py startapp board
```

```
# LearnMart 骨架的等價建立指令 (只供對照)
```

```
uv run django-admin startproject config .
```

```
uv run python manage.py startapp marketplace
```

這些指令只產生骨架，不會自動產生完整商城或留言板。

`urls.py`、`forms.py`、`templates`、示範資料 `command` 都是後續實作。

既有資料夾已建立完成，下一頁改走「同步與啟動」流程。

6-26 從完成專案啟動

在選定的專案根目錄依序執行（兩個專案指令相同）：

```
uv sync
uv run python manage.py migrate
uv run python manage.py seed_demo
uv run python manage.py runserver
```

四行分別執行之步驟與影響：

- **步驟 A** (`uv sync`)：同步 Python 環境與依賴
- **步驟 B** (`migrate`)：建立資料庫 schema
- **步驟 C** (`seed_demo`)：建立課堂示範資料
- **步驟 D** (`runserver`)：啟動本機開發伺服器

不要把 `uv sync` 與 `migrate` 混為一談：前者管套件，後者管資料庫。

6-27 seed_demo 建立哪些資料？

目前專案實作 | `*/management/commands/seed_demo.py`

在沒有同名帳號的新資料庫第一次執行時：

專案	示範帳號與資料
LearnBoard	<code>alice / alice12345</code> 、 <code>bob / bob12345</code> ；4 則留言，其中 1 則是訪客
LearnMart	<code>seller / seller12345</code> 、 <code>buyer / buyer12345</code> ；3 個分類、6 個商品、1 則留言

`get_or_create()` 讓重跑不會持續新增同名示範 rows；若帳號已存在，兩個 command 都不會重設既有 **password**、**role** 或 **email**，所以它們不是 reset command。

常見錯誤： 終端機雖會再次印出課堂 credentials，既有同名帳號仍維持原本資料。這些帳號也仅限本機課堂，不可沿用到公開環境。

6-28 預覽完成專案並記錄結果

執行 `runserver` 後：

```
Starting development server at http://127.0.0.1:8000/
```

- 瀏覽器開啟該網址會看到 LearnBoard 留言牆或 LearnMart 商品頁
- 終端機保持被伺服器占用，按 `Ctrl+C` 停止
- 修改 Python 檔通常會觸發自動重新載入
- 這是開發伺服器，不是正式部署伺服器

`127.0.0.1` 代表自己的電腦；`8000` 是 port。

6-29 URL、View 與反向解析的實際對照

項目	LearnBoard	LearnMart
根 URL	<code>include("board.urls")</code>	<code>include("marketplace.urls")</code>
<code>app_name</code>	<code>board</code>	<code>marketplace</code>
首頁 route	<code>""</code>	<code>""</code>
首頁 View	<code>MessageListView.as_view()</code>	<code>ProductListView.as_view()</code>
首頁名稱	<code>board:list</code>	<code>marketplace:home</code>

兩者根 URL 都保留 admin、登入與登出路由。

`as_view()` 把 class-based View 轉成可呼叫的 View，後續章節再深入。

你寫的函式 View 與這些 View，都遵守接收 request、回傳 response 的約定。

6-30 對照動態路由與反向產生網址

LearnMart 的 `marketplace/urls.py` 節錄：

```
path("products/<int:pk>/", views.ProductDetailView.as_view(),  
     name="product-detail")
```

```
reverse("marketplace:home") # '/'  
reverse("marketplace:product-detail", kwargs={"pk": 3}) # '/products/3/'
```

```
{% url 'marketplace:product-detail' product.pk %}
```

LearnBoard 也以 `<int:pk>` 定位要編輯或刪除的留言。

成功產生網址不保證那筆資料存在，物件不存在時仍可能回應 404。

6-31 兩個專案提供的真實案例

來源	本課資料層的對照重點
LearnBoard 原第 4 章	Message 文字、作者、時間、排序、migration
LearnBoard 的 admin.py	留言列表、作者搜尋、短內容顯示
LearnMart 原第 3～5 章	商品圖片、分類與賣家關聯、唯一約束
LearnMart 的 admin.py	商品管理、訂單 Inline、自訂 UserAdmin

一對一 Profile、多對多 Tag、索引與部分 Admin 客製為**教學延伸**。
兩個專案的業務 models 未宣告這些 Profile／Tag 關聯。

6-32 先學 Django，再對照專案

已完成 catalog 最小資料練習；接下來逐項解釋，再對照 LearnBoard 與 LearnMart。

- **基本必學**：Model、欄位、三種關聯、Meta、CRUD、migration、Admin。
- **教學範例**：用留言、商品說明規則；不是只能處理這兩種資料。
- **進階延伸**：索引設計、through、批次操作、併發與權限客製。
- **閱讀方法**：先問用途，再讀語法，執行後觀察結果，最後解釋原因。

尚未懂的英文術語會在第一次使用時定義；不要先背整份 API 清單。

6-33 切換案例前先確認環境

你正在操作哪個專案？	匯入 models 的位置	是否需要現成資料？
最小練習 django_lab	catalog.models	前面自行建立
LearnBoard	board.models	依專案初始化流程
LearnMart	marketplace.models	依專案初始化流程

之後標示「現有 LearnMart」的範例，需切到 learnmart 的 manage.py 所在目錄。語法相同不代表模型欄位完全相同；不要跨專案混用 import 或資料庫。

已定義完整 class 的例子可建模；標示「節錄」的例子用來讀語法，不是完整檔案。

第 6 章實作與離堂檢核

任務： 在既有 django_lab 加入 catalog，建立分類與商品並在 Admin 找到資料。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： LearnBoard 01 原第 4 章；LearnMart 01 原第 4 章。手冊保留原章號，對照表列出本課位置。

第 7 章

欄位型別與資料規則

提出欄位設計表，說明型別、空值、預設、唯一性與圖片存放位置。

本章的操作環境與成果

catalog 為完整帶做範例；board／marketplace 節錄需切到對應專案。Profile／Tag 等為教學延伸。

完成成果： 提出欄位設計表，說明型別、空值、預設、唯一性與圖片存放位置。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。進階頁可回查，但所有基本驗收需完成。

7-1 設計欄位先問五個問題

1. 這是文字、數字、日期，還是另一個物件？
2. 沒填時代表什麼：空字串、未知，還是預設值？
3. 值可否重複？需要哪種範圍與格式限制？
4. 誰可以輸入或修改：使用者、管理員，還是程式？
5. 哪些規則需要資料庫也強制保護？

練習：電話看起來都是數字，為何仍用 `CharField` ？
要保留前導 0、加號與分機，而且不拿來做加減乘除。

7-2 常用欄位選擇表

資料需求	Django 欄位	例子
短文字／長文字	CharField／TextField	名稱／留言
整數／精確小數	IntegerField／DecimalField	庫存／金額
測量近似值／真假	FloatField／BooleanField	感測值／上架
日期／時間／日期時間	DateField／TimeField／DateTimeField	生日／開門／下單
網址代稱／圖片	SlugField／ImageField	tech／商品圖

先看資料意義，再決定型別；不要全部存成字串。

7-3 讀懂一個欄位宣告

教學延伸 | 修改 Product.name 的說明文字

```
name = models.CharField(  
    "商品名稱", max_length=150,  
    help_text="請填可辨識的名稱，不超過 150 字元。",  
)
```

- `name`：Python 與 ORM 使用的名稱。
- `CharField`：資料型別；`max_length`：長度設定。
- 第一個參數是 `verbose_name`：表單／Admin 的人類可讀名稱。
- `help_text`：輸入提示；它不會取代 validator。

ForeignKey 第一個參數是關聯目標，中文標籤改用 `verbose_name=`。

7-4 CharField 與 TextField

抽入 LearnBoard 留言與 LearnMart 商品案例

```
# Product
name = models.CharField("商品名稱", max_length=150)
description = models.TextField("商品說明")
# Message
content = models.TextField("留言內容", max_length=500)
```

短文字通常用 CharField；多行內容通常用 TextField。

TextField 的 max_length 會影響產生的表單，但不等於資料庫長度限制，也不能假設直接呼叫 Model.full_clean() 就會替 TextField 擋住超長文字。

要在 model 驗證長文上限，可明確加 `MaxLengthValidator(500)`。

7-5 blank 與 null：兩個不同層次

參數	管理的層次	問題
blank=True	Django 表單／模型驗證	可以不輸入嗎？
null=True	資料庫	可以存 SQL NULL 嗎？

```
note = models.CharField(max_length=100, blank=True)
shipped_at = models.DateTimeField(null=True, blank=True)
```

文字通常用空字串表示未填；未出貨的時間則用 NULL。

`blank=False` 不是資料庫「禁止空字串」的約束。

可選又唯一的文字欄位是特例：多個空值需另思考 NULL 與唯一性。

7-6 default、editable 與驗證

```
stock = models.PositiveIntegerField(default=0)
```

- `default` 在建立物件未提供值時使用，不是自動修正非法輸入。
- `callable` 預設值寫函式本身；可變物件如 JSON 預設值用 `dict`。
- `editable=False` 讓欄位不出現在自動表單；程式仍可修改資料。
- `db_default` 是資料庫層預設；不要把普通 `default` 當成 SQL DEFAULT。

7-7 full_clean 與 save 分開理解

```
product.full_clean() # 明確做模型驗證  
product.save()      # 儲存；本身不自動呼叫 full_clean()
```

驗證器 (validator) 判斷值是否符合規則；失敗會產生 `ValidationError`。
模型驗證還會檢查唯一性與約束；完整流程在第 9 章展開。

練習：如果只用 `save`，價格小數位數是否一定會先得到清楚的輸入錯誤？
不能保證。輸入驗證與資料庫寫入是不同步驟。

7-8 整數欄位：0 算不算合法？

```
stock = models.PositiveIntegerField(default=0)
quantity = models.PositiveIntegerField(
    default=1, validators=[MinValueValidator(1)],
)
```

第一行是商品庫存；後三行是 CartItem 數量，需先匯入 validator。

- IntegerField 可存正負整數；PositiveIntegerField 包含 0。
- 「非負」與「至少 1」不同，因此購物數量另加最小值驗證。
- Small／Big 版本調整整數範圍；不要靠名稱猜資料庫容量。
- validator（驗證器）提供輸入錯誤訊息；DB 防線另看 CheckConstraint。

7-9 DecimalField 與 FloatField

```
from decimal import Decimal
```

```
price = models.DecimalField(max_digits=8, decimal_places=2)  
# Python 金額使用 Decimal("199.90")
```

8 是總位數，2 是小數位數，因此整數部分最多 6 位。

FloatField 是二進位近似值，適合能接受誤差的測量資料。

現有 LearnMart 使用 `max_digits=10, decimal_places=0`，不含小數。

Decimal 避免 Python 端浮點誤差；SQLite 的小數儲存／運算仍有限制，不能把欄位宣告等同於所有資料庫都保證精度與範圍。

7-10 Boolean 與 choices

現有 LearnMart | User 的角色宣告，節錄

```
class Role(models.TextChoices):  
    BUYER = "buyer", "買家"  
    SELLER = "seller", "賣家"  
  
role = models.CharField(  
    max_length=10, choices=Role.choices, default=Role.BUYER,  
)
```

真假狀態用 BooleanField；多種有限狀態用 choices。

資料存 seller；user.get_role_display() 得到「賣家」。

choices 是驗證與選單設定；不自動成為資料庫 CHECK 或狀態轉移流程。

7-11 日期、時間、日期時間

型別	Python 值	適合需求
DateField	date	生日、截止日期
TimeField	time	每日開門時間
DateTimeField	datetime	留言建立、出貨時刻
DurationField	timedelta	持續多久

生日不需要時分秒；交易發生的時刻通常需要日期與時間。

別把格式化的「2026/09/19 08:00」存成 CharField。

格式留給顯示層，資料保留可查詢與比較的型別。

7-12 三種時間設定怎麼選？

設定	行為	適合欄位
<code>auto_now_add=True</code>	首次 save 自動填入	建立時間
<code>auto_now=True</code>	save 寫入該欄位時更新	修改時間
<code>default=timezone.now</code>	建立時提供預設，可自行指定	發佈／事件時間

```
from django.utils import timezone
published_at = models.DateTimeField(default=timezone.now)
```

同一欄位不要混用這三種選項。auto_now 與 auto_now_add 會設為不可編輯。

`QuerySet.update()` 不觸發 auto_now；`update_fields` 也要包含時間欄位。

7-13 時區與未發生的事件

現有 LearnMart | 尚未出貨時不應捏造出貨時間

```
shipped_at = models.DateTimeField(null=True, blank=True)
```

- 啟用 `USE_TZ` 時，使用 `timezone.now()` 取得 aware datetime。
- 資料儲存與畫面顯示時區分開處理；不要自行加 8 小時再存回去。
- `TIME_ZONE` 影響預設顯示／解讀；不是字串格式設定。
- 未出貨用 `None`；查詢用 `shipped_at__isnull=True`。

練習：建立時間、生日、未出貨時間，哪個需要 null？為什麼？

7-14 SlugField：可讀的網址代稱

現有 LearnMart | Category

```
slug = models.SlugField("網址代稱", max_length=80, unique=True)
```

- 例如 `tech`、`daily-tools`；預設接受 ASCII 字母、數字、底線與連字號。
- `allow_unicode=True` 才允許 Unicode 字母，例如中文。
- SlugField 不會自動由 name 產生內容，也不預設唯一。
- 目前商城以 query string 傳分類 slug；商品詳情仍使用 pk。

「有 slug 欄位」不代表 URLconf 已經使用它。

7-15 Slug 生成與重複值

教學延伸

```
from django.utils.text import slugify
slugify("Daily Tools")          # daily-tools
slugify("生活用品", allow_unicode=True)
```

slugify 只做字串轉換；不同名稱可能得到相同 slug。

若網址要求唯一，需設計衝突策略，並以資料庫唯一約束守住競態。

Admin 可用 `prepopulated_fields` 輔助輸入，但不是所有寫入入口的生成器。

網址若採內建 `<slug:slug>` converter，不能直接假設它接受中文。

7-16 圖片：資料庫存路徑，storage 存檔案

抽入 LearnMart 商品圖片案例

```
image = models.ImageField(
    "商品圖片", upload_to="products/%Y/%m/", blank=True,
)
```

- 欄位保存檔名／路徑參照；實際內容交給 storage。
- `upload_to` 相對於 storage，不是完整的公開網址。
- `.name` 是儲存名稱；`.url` 是存取網址；`.path` 不保證遠端 storage 支援。
- ImageField 需要 Pillow；驗證圖片不等於所有上傳安全問題都已解決。

7-17 圖片從上傳到顯示

```
{% if product.image %}
  
{% else %}
  <p>尚無商品圖片</p>
{% endif %}
```

- 先檢查有沒有檔案，再讀 url；空圖片直接讀 url 會出錯。
- 開發環境分清 `MEDIA_ROOT` 與 `MEDIA_URL`。
- 自訂上傳表單需要 `multipart` 與 `request.FILES`，接續第 15 章。
- 備份需包含 DB 與 media；刪除 model 不會自動清除 storage 的檔案。

7-18 欄位設計練習（20 分鐘）

請替「校內二手物品」填一份欄位設計表：

名稱、說明、價格、數量、上架狀態、網址代稱、圖片、建立時間、成交時間。

每個欄位寫出：**型別、必要參數、可否空白、預設值與理由**。

驗收：

- 說明價格不用 float，數量是否允許 0。
- 說明成交時間為何不能一律用 `auto_now_add`。
- 指出哪個是 primary key，slug 的 unique 又保護什麼。
- 同桌交換，找出一項「只在表單驗證，DB 未必保證」的規則。

第 7 章實作與離堂檢核

任務： 提出欄位設計表，說明型別、空值、預設、唯一性與圖片存放位置。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 01 原第 4 章](#)；[LearnMart 01 原第 4 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[05_模型關聯與Migration](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。
- 原始教材與合併去向見 [來源索引](#)。